Cut y Negación

Curso Programación Lógica 2025

cut

El cut elimina puntos de opción en el árbol SLD.

Por ejemplo, para una regla de la forma

```
q:- p1, … , pn, !, r1, ... , rn.
```

el cut deja fijas las opciones en cuanto a:

- esta cláusula particular de `q`
- `p1, ... , pn`

NO afecta el funcionamiento en cuanto a `r1, ... , rn`.

cut

IMPORTANTE

- Incluir un cut en un predicado puede modificar el conjunto de soluciones.
- Si eso sucede se le llama cut rojo.
- Si el cut no modifica las soluciones posibles, se le llama cut verde.
- El cut verde se usa para mejorar la eficiencia, y es el uso recomendado para el cut.

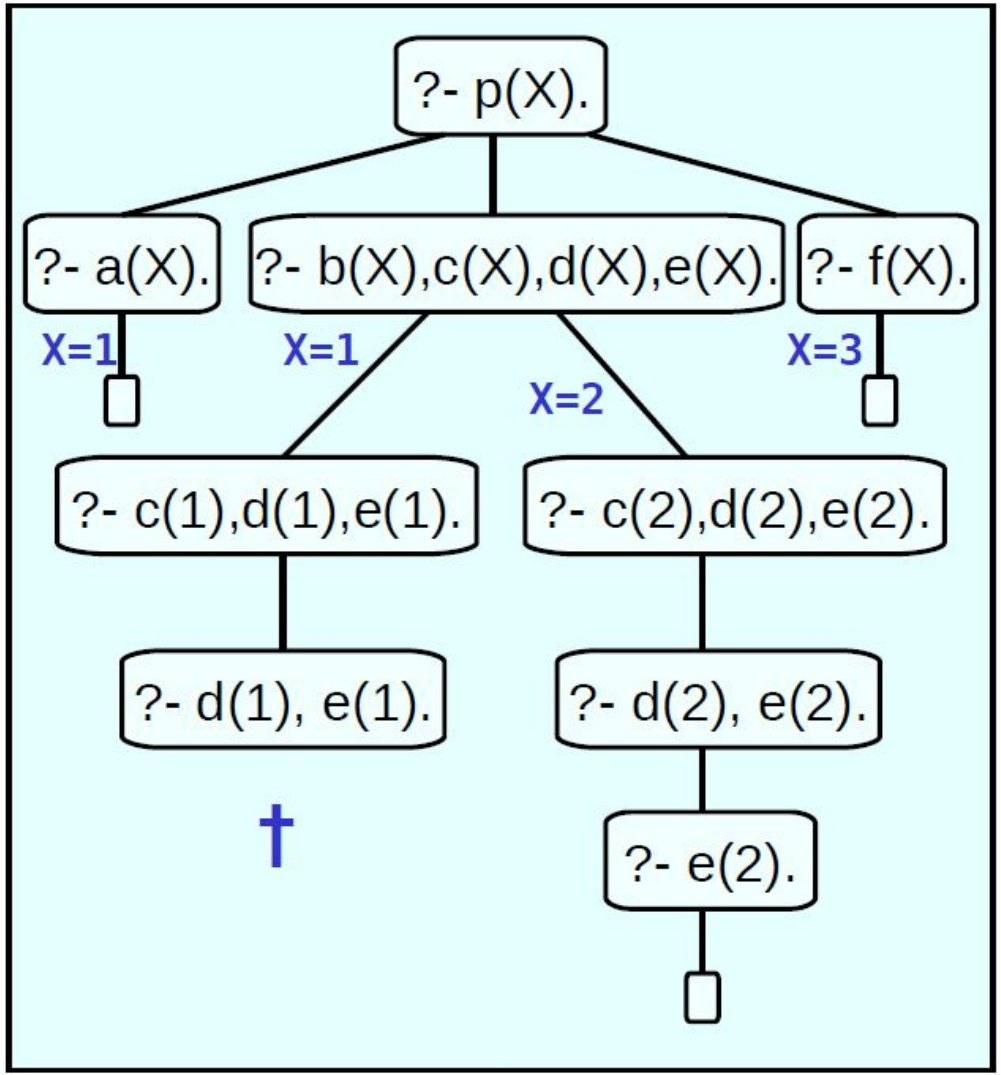
```
p(X):- a(X).
p(X):- b(X), c(X), d(X), e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

```
p(X):- a(X).
p(X):- b(X), c(X), d(X), e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

```
?- p(X).
```

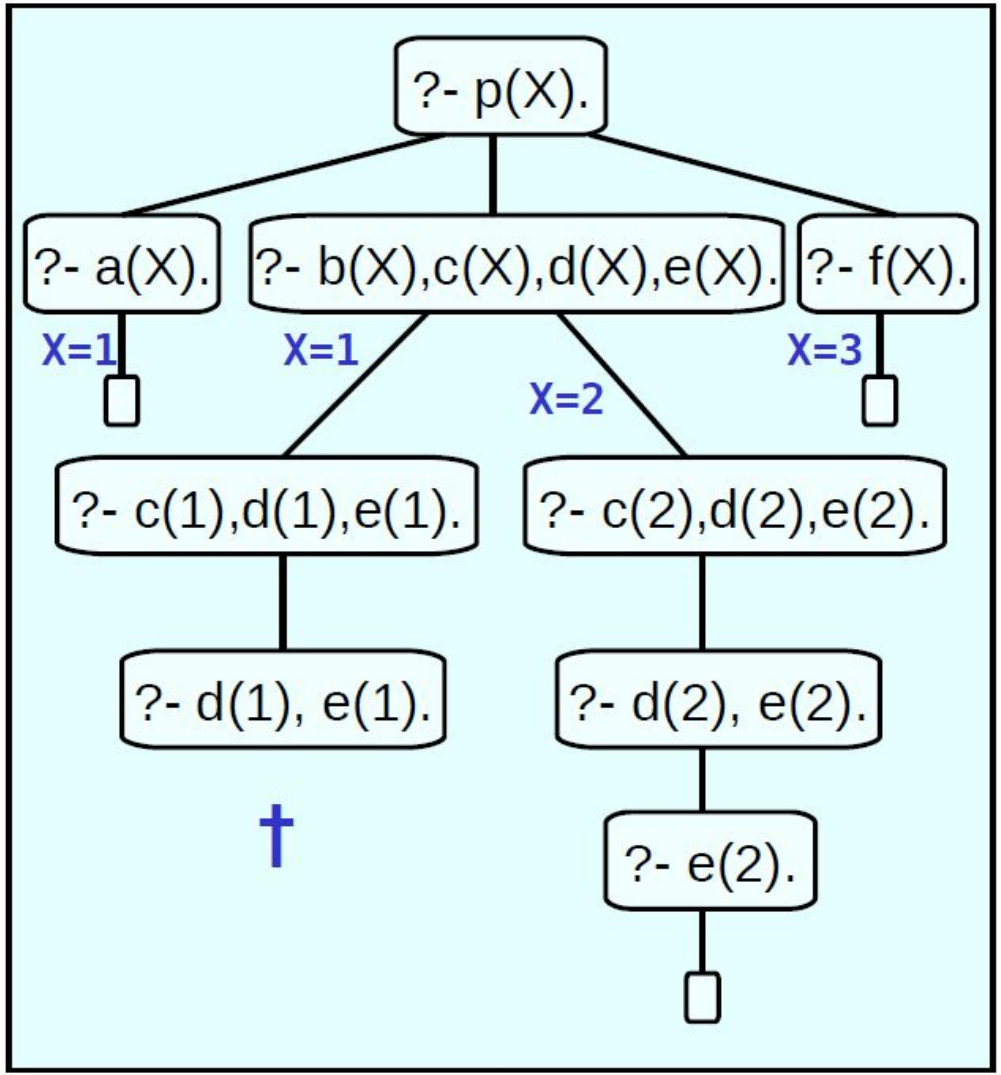
```
p(X):- a(X).
p(X):- b(X), c(X), d(X), e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

```
?- p(X).
```



```
p(X):- a(X).
p(X):- b(X), c(X), d(X), e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

```
?- p(X).
X=1;
X=2;
X=3;
no
```



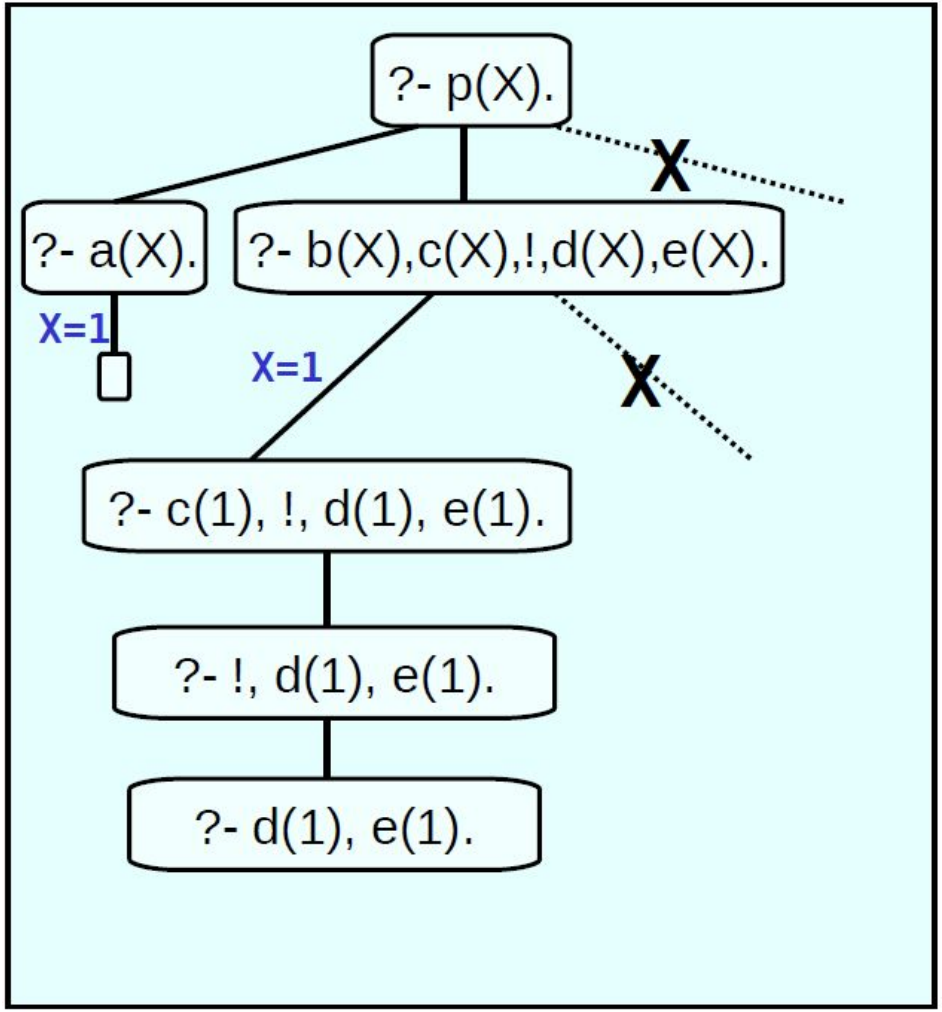
```
p(X):- a(X).
p(X):- b(X),c(X),!,d(X),e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

```
p(X):- a(X).
p(X):- b(X),c(X),!,d(X),e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

```
?- p(X).
```

```
p(X):- a(X).
p(X):- b(X),c(X),!,d(X),e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

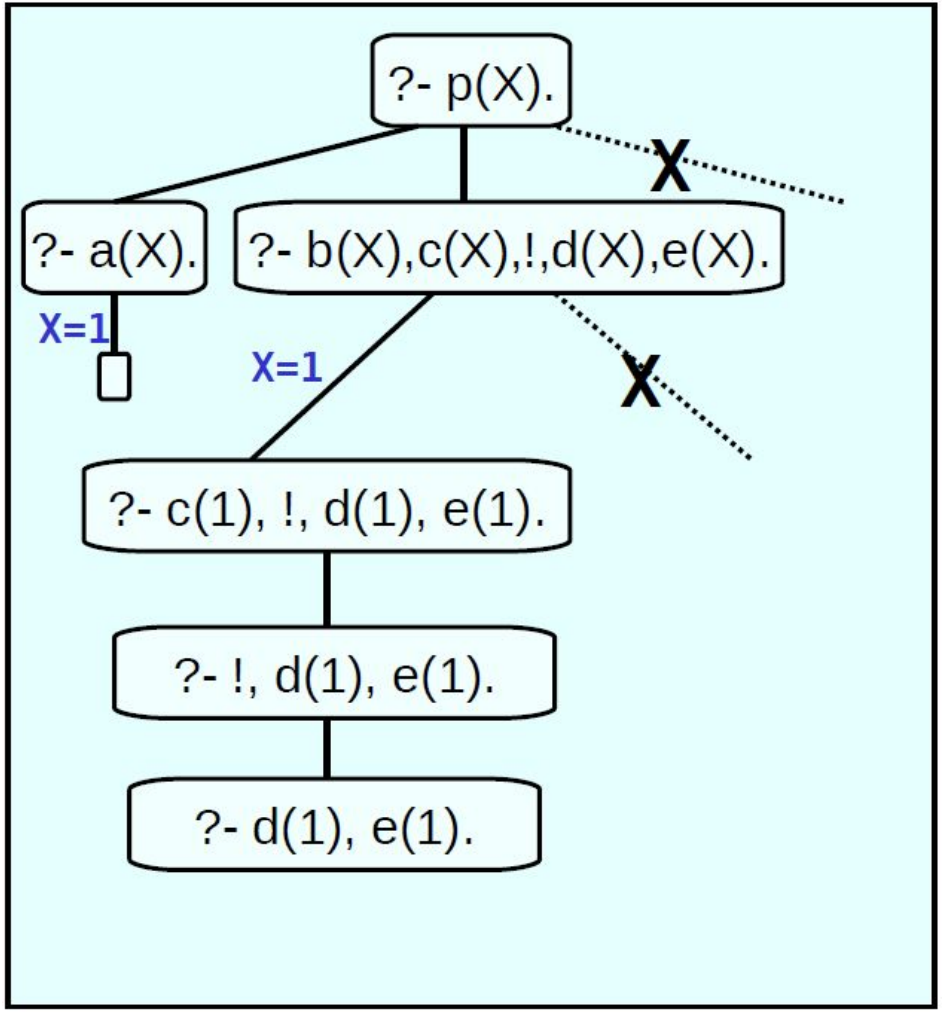
```
?- p(X).
```



```
p(X):- a(X).
p(X):- b(X),c(X),!,d(X),e(X).
p(X):- f(X).
a(1).
b(1).   b(2).
c(1).   c(2).
d(2).
e(2).
f(3).
```

¿cut rojo o verde?

```
?- p(X).
X=1;
no
```



Ejercicio hecho en clase

Dado el siguiente programa lógico:

```
s(X,Y) :- p(X,Y).
s(0,0).
p(X,Y):- i(X), j(Y).
p(0,0).
i(1).
i(2).
j(1).
j(2).
j(3).
```

i) Escriba el árbol SLD resultante de hacer la consulta <-s(X,Y)

ii) Muestre cómo cambia el árbol si cambiamos la regla p(X,Y):- i(X),j(Y) por las siguientes variantes:

a) p(X,Y):- i(X), !, j(Y).

b) p(X,Y):- !, i(X), j(Y).

c) p(X,Y):- i(X), j(Y), !.

Negación por falla

not

- Predicado de segundo orden, se aplica a un objetivo.
- `not(G)` es verdadero si G no se puede deducir de nuestra base de conocimientos.
- Esto es distinto a afirmar que G no sea verdadero.
- Si hay ramas infinitas en los árboles de derivación, no es posible afirmar que algo no se puede demostrar.
- La regla de negación por falla finita dice que si NINGUNA de las ramas del árbol me conduce a una refutación, puedo negar el objetivo.

not

```
negacion(Goal):- Goal,!,fail.

negacion(Goal).
```

Observación:

- `fail` es un predicado que siempre falla.
- Es útil para forzar un backtracking.

not

```
not_member_check(X,Xs) :- not(member(X,Xs)).
```

equivalente a

```
not_member_check(X,Xs) :- \+ member(X,Xs).
```

not

Considere que C1, C2 y C3 son conjuntos representados como listas sin elementos repetidos. Implemente los siguientes predicados en Prolog utilizando not:

i. diferencia(+C1,+C2,?C3) ← C3 es el conjunto C1 - C2.
ii. disjuntos(+C1,+C2) ← C1 y C2 son disjuntos.

not

```
disjuntos(C1,C2) :- \+ (member(X,C1),member(X,C2)).
```

Referencias

- Capítulo 11 The Art of Prolog

- Capítulo 10 Learn Prolog Now!